Análisis Caso DevSecOps

Contexto general

Una plataforma de comercio electrónico no solo debe funcionar bien, también debe estar pensada para resistir ataques comunes desde el diseño. Si la empresa desarrolla una aplicación para crear cuentas, buscar y comprar productos, gestionar pedidos y hacer pagos en línea, entonces la seguridad tiene que entrar desde el inicio del ciclo de vida, no al final como un parche.

Desde lo visto en clase, esto encaja muy bien con la idea de DevOps, CI/CD, automatización, trazabilidad y control de cambios. En otras palabras: si el desarrollo se mueve rápido, la seguridad también tiene que moverse rápido, pero sin improvisar.

1. Creación de cuentas

Problemas de seguridad posibles

- **Registro de usuarios con datos falsos o automatizados:** un atacante puede crear muchas cuentas para fraude, spam o abuso de promociones.
- **Contraseñas débiles:** si el sistema no exige buenas políticas, los usuarios terminan usando claves fáciles de adivinar.
- **Exposición de datos personales:** nombre, correo, teléfono o documentos pueden filtrarse si hay malas validaciones o consultas inseguras.
- **Ataques de fuerza bruta o credencial stuffing:** si el login no tiene protecciones, un atacante puede probar credenciales robadas masivamente.
- **Secuestro de cuenta:** si el flujo de recuperación de contraseña es débil, alguien puede tomar control de la cuenta.

Estrategias de mitigación

- **Validación fuerte de entrada y salida:** validar correo, teléfono, contraseña y cualquier dato sensible desde backend, no solo desde el frontend.
- **Políticas de contraseña robustas:** exigir longitud mínima, complejidad razonable y bloqueo de contraseñas comunes.
- **Hash seguro de contraseñas:** almacenar claves con algoritmos resistentes como bcrypt, scrypt o Argon2, nunca texto plano.
- **MFA o autenticación de doble factor:** agregar una capa extra reduce mucho el impacto de robo de credenciales.

- **Rate limiting y CAPTCHA:** frena automatización y ataques masivos de registro o login.
- **Verificación de correo o teléfono:** ayuda a confirmar que la cuenta corresponde a un usuario real.
- **Recuperación de cuenta segura:** enlaces temporales, tokens de un solo uso y expiración corta.

Ventajas

Estas prácticas reducen la superficie de ataque desde el arranque. Si las contraseñas se guardan bien, aunque haya una fuga no se expone todo tan fácil. Si se limita el número de intentos y se valida el origen de la cuenta, se le complica mucho la vida al atacante. En seguridad, hacer más difícil el abuso ya es una gran victoria.

2. Búsqueda y compra de productos

Problemas de seguridad posibles

- **Inyección SQL o consultas mal construidas:** si la búsqueda no está bien parametrizada, pueden acceder o alterar información de la base de datos.
- **Manipulación de precios o carrito:** un usuario malicioso puede cambiar valores desde el navegador y pagar menos de lo debido.
- **Cross-Site Scripting (XSS):** un atacante puede insertar scripts en campos de búsqueda, comentarios o filtros.
- **Acceso no autorizado a productos o datos internos:** catálogos, descuentos o reglas internas pueden quedar expuestos.
- **Fraude en la compra:** uso de tarjetas robadas, bots comprando rápido o abuso de cupones.

Estrategias de mitigación

- **Consultas parametrizadas y ORM seguro:** evitar concatenar strings en SQL.
- **Validación en servidor de precios y totales:** nunca confiar en valores enviados por el cliente para calcular el cobro final.
- **Sanitización y codificación de salida:** neutralizar contenido que pueda ejecutarse como script en el navegador.

- **Control de acceso por roles y permisos:** separar lo que ve un cliente normal de lo que ve un administrador u operador.
- **Firmar o recalcular el carrito en backend:** así se evita que el usuario altere el monto final.
- **Monitoreo antifraude:** detectar patrones raros como muchas compras en poco tiempo o comportamientos automatizados.
- **Revisiones de código y pruebas automatizadas de seguridad:** incluir pruebas contra inyección, XSS y validaciones de negocio.

Ventajas

La idea aquí es no confiar en lo que venga del navegador. El cliente puede mentir, tocar campos y alterar tráfico; el backend es el que debe mandar. Si el precio real se calcula del lado del servidor y el código usa consultas seguras, se evita una buena parte del fraude y de la manipulación del sistema.

3. Gestión de pedidos

Problemas de seguridad posibles

- **IDOR o acceso indebido a pedidos ajenos:** si alguien cambia un identificador de pedido, podría ver información de otro cliente.
- **Alteración del estado del pedido:** un atacante podría intentar marcar un pedido como pagado, enviado o entregado sin autorización.
- **Filtrado de información sensible:** dirección, teléfono, historial de compras o detalles logísticos pueden quedar expuestos.
- **Problemas de trazabilidad:** si no hay logs claros, es difícil saber quién cambio un pedido y cuando.
- **Manipulación de devoluciones o cancelaciones:** procesos mal protegidos pueden ser usados para fraude.

Estrategias de mitigación

- **Autorización por objeto y por rol:** verificar que el usuario tenga permiso para ver o modificar ese pedido en específico.
- **Uso de identificadores opacos o no secuenciales:** disminuye la posibilidad de adivinar pedidos ajenos.

- **Registro de auditoria:** guardar quien hizo el cambio, desde donde y en qué momento.
- **Estados de pedido controlados por reglas de negocio:** solo permitir transiciones validas, por ejemplo, no pasar de creado a entregado de forma directa.
- **Separación de responsabilidades:** el cliente no debe poder forzar estados internos; el backend debe decidir los cambios.
- **Pruebas automatizadas sobre reglas de negocio:** validar que un pedido no pueda ser alterado fuera del flujo esperado.

Ventajas

La gestión de pedidos no solo es un tema funcional, también es de confianza. Si un usuario puede tocar pedidos ajenos o saltarse estados, el sistema pierde integridad. Con autorización bien aplicada, reglas claras y auditoria, el proceso queda más controlado y mucho más fácil de investigar si algo sale mal.

4. Pagos en línea

Problemas de seguridad posibles

- **Exposición de datos financieros:** tarjetas, tokens o referencias de pago pueden quedar visibles si la integración está mal hecha.
- **Intercepción de transacciones:** sin protección adecuada, un atacante puede intentar interceptar o manipular la comunicación.
- **Reutilización de tokens o sesiones:** si los tokens viven demasiado o no están bien acotados, se pueden reutilizar.
- **Fraude de pago:** compras con tarjetas robadas o intentos de repetir cobros.
- **Errores de integración con pasarelas:** una mala configuración puede duplicar cargos o confirmar pagos incompletos.

Estrategias de mitigación

- **Uso de pasarelas certificadas:** reducir la cantidad de datos sensibles que toca directamente la aplicación.
- **HTTPS obligatorio y configuración segura de TLS:** proteger la información en tránsito.

- **Tokenización de datos de pago:** evitar almacenar información completa de tarjetas.
- **Cumplimiento de buenas prácticas tipo PCI DSS:** aunque no se haga toda la certificación completa, si conviene seguir sus principios básicos.
- **Validación de callbacks y webhooks:** confirmar que la respuesta realmente venga del proveedor de pagos.
- **Idempotencia en cobros:** evitar cobros duplicados si la petición se repite.
- **Monitoreo de transacciones sospechosas:** detectar montos raros, reintentos excesivos o patrones automatizados.

Ventajas

En pagos, el objetivo no es solo que funcione, sino que sea confiable. HTTPS, tokenización y validación de callbacks ayudan a reducir el riesgo de fuga o manipulación. Y la idempotencia es clave porque en sistemas reales las peticiones pueden repetirse; si no se controla eso, aparecen cobros dobles y caos operativo.

5. Estrategias transversales de desarrollo seguro

Además de lo especifico por función, la plataforma necesita prácticas de base que sostengan toda la seguridad:

- **CI/CD con pruebas de seguridad automatizadas:** integrar pruebas unitarias, de integración y de seguridad en el pipeline.
- **Análisis estático de código:** detectar vulnerabilidades antes de llegar a producción.
- **Control de versiones y revisiones de código:** evita cambios invisibles y mejora la trazabilidad.
- **IaC para infraestructura repetible:** servidores, redes y configuraciones deben definirse como código para reducir errores humanos.
- **Principio de mínimo privilegio:** cada servicio y usuario solo debe tener lo estrictamente necesario.
- **Monitoreo y alertas:** detectar comportamientos anormales a tiempo.
- **Gestión de secretos:** no guardar llaves, tokens ni credenciales en el código fuente.

- **Backups y plan de recuperación:** si algo falla, el sistema debe poder volver rápido.

Ventajas

Esto conecta directamente con lo visto en clase: automatización, trazabilidad, control de cambios y mejora continua. Cuando la seguridad entra al pipeline, el equipo detecta fallos más temprano y a menor costo. Es mucho mejor romper algo en pruebas que descubrirlo después en producción.

6. Conclusión

Una plataforma de comercio electrónico necesita seguridad en cada punto donde el usuario interactúa con el sistema. En cuentas, compras, pedidos y pagos, el riesgo siempre cambia, pero la lógica es la misma: validar, controlar accesos, registrar eventos, automatizar pruebas y no confiar en el cliente.

La idea practica es sencilla: si el desarrollo ya usa cultura DevOps y CI/CD, entonces la seguridad no debe ser un agregado final. Debe ir dentro del proceso. Así la plataforma gana en confianza, integridad y capacidad de respuesta frente a errores o ataques.